

Övning 2: Prolog

Unifiering, negation, listor, träd, permutationer — och två svåra

Marcus Dicander

EECS / Teoretisk datalogi, KTH

DD1351 Logik för dataloger, HT 2026

Uppgifterna är Tomas Sjölands, anpassade av Jesper Amilon. Koden ligger i github.com/dicander/prolog; allt som visas här är kört i SWI-Prolog 9.

1. Unifiering: vad svarar systemet på $X=f(a,Y)$, $X=f(Z,b)$, $Y=Z$?
2. Negation as failure: vad $\backslash +$ betyder, och vad det inte betyder
3. `removeLast/2`: rekursion över listor ovn2.pl
4. Binära träd: representation, `sum/2`, `height/2` bintree.pl
5. `permute/2`: låt backtracking göra jobbet perm.pl
6. Filurien (svår): generera och testa koalitioner filurien.pl
7. Skolschema (svår): databasdesign och predikat i ord scheduling.pl

Poängen

Tre saker återkommer i varje uppgift: unifiering gör allt arbete, backtracking hittar alla svar, och ett predikat är en *relation*, inte en funktion. Den som håller isär de tre klarar tentan.

Predikaten ni får använda, och vad de gör

```
append([], L, L).  
append([H|T], L, [H|R]) :- append(T, L, R).  
  
select(X, [X|T], T).  
select(X, [Y|T], [Y|R]) :- select(X, T, R).  
  
member(X, L) :- select(X, L, _).  
memberchk(X, L) :- select(X, L, _), !.
```

```
?- append(X, Y, [1,2]).  
X = [], Y = [1,2] ;  
X = [1], Y = [2] ;  
X = [1,2], Y = [] .  
?- select(X, [1,2,3], R).  
X = 1, R = [2,3] ;  
X = 2, R = [1,3] ;  
X = 3, R = [1,2] .
```

- `append(A, B, C)`: *C är A följt av B.*
En relation: ge C och få alla sätt att dela upp den. Det använder vi i uppgift 3 och 5.
- `select(X, L, R)`: *X är ett element i L, och R är L utan det elementet.*
Backtracking ger varje element i tur och ordning. Uppgift 5 och 6.
- `member(X, L)`: som `select` men resten slängs. Ger alla element vid backtracking.
- `memberchk`: samma sak med ett snitt (!): första träffen räcker, inga fler svar. Snabbare, men inte längre en relation.

Uppgift 1

Unifiering är inte tilldelning

Uppgift 1: ?- $X=f(a,Y)$, $X=f(Z,b)$, $Y=Z$.

Tre mål i följd. Varje = är en *unifiering*: hitta bindningar som gör båda sidor lika.

Mål	Vad som händer	Bindningar efteråt
$X=f(a,Y)$	X är fri: binds till termen	$X = f(a,Y)$
$X=f(Z,b)$	$f(a,Y)=f(Z,b)$: argument för argument	$Z = a, Y = b$, alltså $X = f(a,b)$
$Y=Z$	$b=a$: två olika konstanter	misslyckas

?- $X=f(a,Y)$, $X=f(Z,b)$, $Y=Z$.
`false.`

Svar

Inga bindningar presenteras alls: systemet svarar `false`. De två första målen binder Y till b och Z till a ; det tredje kräver att b och a är samma term, vilket de inte är. Det finns inga valpunkter att backtracka till (= har bara ett sätt att lyckas), så hela konjunktionen misslyckas.

Uppgift 1: varför det inte är tilldelning

```
?- X=f(a,Y), X=f(Z,b).
```

```
X = f(a, b),
```

```
Y = b,
```

```
Z = a.
```

```
?- Y=Z, X=f(a,Y), X=f(Z,b).
```

```
false.
```

```
?- X=f(a,Y), X=f(Z,b), Y==Z.
```

```
false.
```

- Utan tredje målet får vi bindningarna många *tror* är svaret på uppgiften.
- Ordningen spelar ingen roll för resultatet: Y=Z först gör Y och Z till *samma* variabel, och då kan den inte vara både a och b.
- == testar identitet utan att binda något. Här ger det samma svar, men ?- Y==Z. med fria variabler är falskt medan ?- Y=Z. lyckas.

Vanligt fel

Att läsa Y=Z som "sätt Y till Z" och svara $X=f(a,b)$, $Y=b$, $Z=a$. En bindning i Prolog är permanent under resten av konjunktionen; den kan bara försvinna genom backtracking, och = lämnar ingen valpunkt att backtracka till.

Uppgift 2

Negation as failure

Uppgift 2: negation as failure

Svar

`\+` G lyckas om och endast om Prolog *misslyckas* med att bevisa G (och sökningen tar slut). Prolog kan inte bevisa att något är falskt; det kan bara konstatera att det inte finns något bevis i programmet. Negationen är alltså "kan inte visas", inte "är falskt": *slutna världens antagande* (closed world assumption).

```
% Så här är \+ definierat, i princip:  
\+ G :- call(G), !, fail.  
\+ _.
```

Lyckas G: snittet stänger andra klausulen och `fail` misslyckas. Misslyckas G: första klausulen faller, andra lyckas. Inga bindningar från G överlever.

```
?- \+ member(3, [1,2]).  
true.  
?- \+ member(X, [1,2]).  
false.  
?- member(X, [1,2,3]), \+ X = 2.  
X = 1 ;  
X = 3.  
?- \+ X = 2, member(X, [1,2,3]).  
false.
```

Uppgift 2: vad som går fel

- $\backslash + \text{member}(X, [1,2])$ är falskt fastän det finns massor av X som inte ligger i listan: Prolog *hittar* ett X (nämligen 1) som gör målet sant, alltså misslyckas negationen. Fri variabel i $\backslash +$ betyder "finns det något?", inte "gäller det för alla?".
- Därför spelar ordningen roll: $\backslash + X = 2$ före `member` frågar om X *kan* bli 2 (ja), efter `member` frågar den om *det här* X är 2. Regel: negera bara mål vars variabler redan är bundna.
- Databasen är sluten: $\backslash + \text{flyger}(\text{pingvin})$ är sant om ingen skrivit att pingviner flyger, även om ingen skrivit något alls om pingviner.

	Logisk negation $\neg p$	Negation as failure $\backslash + p$
betyder	p är falskt	p går inte att bevisa ur programmet
kräver	sanningsvärde hos p	att sökningen efter p tar slut
med fri variabel	$\neg \exists X. p(X)$ eller $\exists X. \neg p(X)$?	alltid "finns inget bevis för något X "
bindar variabler	—	aldrig

Uppgift 3

removelast/2

```
list([]).  
list([H|T]) :- list(T).  
  
removelast([], []).  
removelast([H|T], [H|R]) :-  
    removelast(T, R).
```

```
?- removelast([1,2,3], R).  
R = [1,2] ;  
false.  
?- removelast([7], R).  
R = [] ;  
false.  
?- removelast([], R).  
false.  
?- removelast(L, [1,2]).  
L = [1,2,_].
```

- Följ listdefinitionen: ett fall per klausul. Basfallet är inte den tomma listan, för den har inget sista element att ta bort, utan listan med *ett* element.
- Rekursionen behåller huvudet och tar bort sista elementet i svansen.
- `removelast([], R)` misslyckas. Det är rätt: relationen är inte definierad där.
- Det extra `false` efter `;` [3] matchar båda klausulerna, och den andras kropp misslyckas på `[]`. Relationen är värdbar: sista svaret.

Uppgift 3: spårning, och ett alternativ

```
removelast([1,2,3], R)
  klausul 2: H=1, T=[2,3], R=[1|R1]
  removelast([2,3], R1)
    klausul 2: H=2, T=[3], R1=[2|R2]
    removelast([3], R2)
      klausul 1: R2=[]
  R = [1|[2|[[]]]] = [1,2]
```

Svaret byggs uppifrån under nedstigningen:
[1|R1] finns innan R1 är känd. Det är
unifieringens förtjänst, inte något "return".

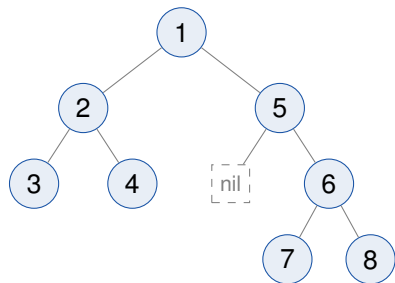
```
% Samma relation med append/3:
removelast(L, R) :- append(R, [_], L).
```

- "L är R följt av en lista med exakt ett element." Ingen egen rekursion; append går igenom alla uppdelningar och bara den med ett element i svansen passar.
- Kortare, men gör mer arbete: append provar varje delning av L.
- Båda versionerna ger `false` för `[]`.

Uppgift 4

Binära träd

Uppgift 4a: representation



Data är termer. Tre sorters delträd, tre funktorer:

```
nil                                % tomt träd
leaf(V)                            % löv med värde V
branch(V, Left, Right)             % nod med två delträd

% Exempelträdet:
branch(1, branch(2, leaf(3), leaf(4)),
      branch(5, nil, branch(6, leaf(7), leaf(8))))
```

- Noden 5 har bara ett högerbarn: vänster blir nil. Ingen deklaration behövs; formen bestäms av att predikaten har en klausul per funktor.
- Alternativ: bara nil och node(V,L,R), där ett löv är node(V,nil,nil). Färre fall per predikat, fler nil.

Uppgift 4b–c: sum/2 och height/2

```
treesum(nil, 0).
treesum(leaf(V), V).
treesum(branch(V, L, R), S) :-
    treesum(L, SL),
    treesum(R, SR),
    S is V + SL + SR.

max(X, Y, X) :- X >= Y.
max(X, Y, Y) :- X < Y.

treeheight(nil, 0).
treeheight(leaf(_), 1).
treeheight(branch(_, L, R), H) :-
    treeheight(L, HL),
    treeheight(R, HR),
    max(HL, HR, M),
    H is M + 1.
```

```
?- treesum(T, S).           % T = exempelträdet
S = 36.
?- treeheight(T, H).
H = 4.
```

- Strukturell rekursion: en klausul per funktor, delträden först, sedan räkna. Samma mönster som för listor, fast med två svansar.
- `is` behövs för aritmetik; $S = V + SL + SR$ skulle bara bygga termen $1+3+4$ utan att räkna ut den.
- `max/3` skrivs med två klausuler och ömsesidigt uteslutande villkor, så att inget snitt behövs.

Uppgift 4: två anmärkningar

Snitten i `bintree.pl`

Repots version har ett `!` efter varje klausulhuvud, till exempel `treesum(leaf(X), X) :- !`. De ändrar inga svar: funktorerna `nil`, `leaf` och `branch` utesluter redan varandra, och SWI-Prolog indexerar på första argumentet, så det finns ingen valpunkt att skära bort. Gröna snitt: harmlösa men överflödiga. Ett snitt som *behövs* för rätt svar är rött, och det ska ni misstro.

Vad "höjd" betyder

Här räknas noder på den längsta vägen: ett löv har höjd 1, exemplet höjd 4 (1-5-6-7). Räknar man kanter, som uppgiftens "avstånd" antyder, är lövets höjd 0 och svaret 3: byt basfallet till `treeheight(leaf(_), 0)`. Poängen är inte vilket ni väljer utan att ni säger vilket.

Uppgift 5

permute/2

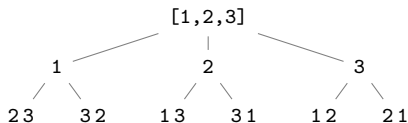
Uppgift 5: permute/2 med select

```
permute([], []).  
permute(L, [X|P]) :-  
    select(X, L, R),  
    permute(R, P).
```

```
?- permute([1,2,3], F).  
F = [1,2,3] ;  
F = [1,3,2] ;  
F = [2,1,3] ;  
F = [2,3,1] ;  
F = [3,1,2] ;  
F = [3,2,1] ;  
false.
```

[1,2,3,4] ger 24 svar, det första [1,2,3,4], sedan [1,2,4,3].

- Läs klausulen som en definition: en permutation av L är något element X ur L följt av en permutation av resten.
- *select väljer* inte ett element; det ger ett i taget vid backtracking. Därför kommer alla $n!$ permutationerna gratis när användaren trycker ;.



sökträdet: varje nivå ett *select*, varje löv ett svar

Uppgift 5: repots variant, med append

```
% perm.pl
permutation([], []).
permutation([E|Xs], Ys) :-
    permutation(Xs, Ps),
    append(Front, Back, Ps),
    append(Front, [E|Back], Ys).
```

```
?- permutation([1,2,3], F).
F = [1,2,3] ;
F = [2,1,3] ;
F = [2,3,1] ;
F = [1,3,2] ;
F = [3,1,2] ;
F = [3,2,1] ;
false.
```

- Andra vinkeln: permutera svansen först, stoppa sedan in huvudet E på *varje* möjlig plats.
- Första append delar Ps i alla par (prefix, suffix); andra bygger ihop dem med E emellan. Här används append baklänges: från helhet till delar.
- Samma sex svar, i annan ordning. Båda versionerna är korrekta; välj den ni kan förklara.
- Med första argumentet obundet loopar båda efter sista svaret: den vanliga reservationen mot "relationer i alla riktningar".

Uppgift 6

Filurien (svår)

Uppgift 6: modellen

```
party(p1,38). party(p2,17). party(p3,51).
party(p4,23). party(p5,27). party(p6,35).
party(p7,18). party(p8,25).
parties([p1,p2,p3,p4,p5,p6,p7,p8]).

total_seats(T) :-
    findall(N, party(_,N), Ns),
    sum_list(Ns, T).          % 234

% Gemensamhetsindex, bara för A @< B:
s2(p1,p2, 2). s2(p1,p3,-6). s2(p1,p4, 5).
s2(p1,p5, 4). s2(p1,p6, 6). s2(p1,p7,-1).
s2(p1,p8, 1). s2(p2,p3,-7). ... % 28 fakta

sim(P, P, 0).
sim(A, B, S) :- A @< B, !, s2(A, B, S).
sim(A, B, S) :- A @> B, s2(B, A, S).
```

- Mandaten är fakta. Totalen räknas fram med `findall + sum_list` i stället för att skrivas in: en siffra mindre att ha fel.
- Matrisen är symmetrisk, så bara den övre triangeln lagras: 28 fakta i stället för 64. `sim/3` vänder på paret vid behov; `@<` jämför atomer i standardordning.
- Diagonalen är 0: ett parti har inget index mot sig självt.
- Uppgiften säger "hitta på lämpliga värden själva". Värdena här är påhittade så att det finns stabila regeringar utan att alla är det.

Uppgift 6: generera och testa

```
subset_of([], []).
subset_of([H|T], [H|R]) :- subset_of(T, R).
subset_of([_|T], R)      :- subset_of(T, R).
pairwise_score(Coal, Score) :-
    findall(S, (member(A,Coal),
                member(B,Coal), A @< B,
                sim(A,B,S)), Ss),
    sum_list(Ss, Score).
majority(Coal) :-
    seats_sum(Coal, S), total_seats(T),
    2*S >= T.           % minst hälften
stable(Coal, Seats, Score) :-
    parties(All), subset_of(All, Coal),
    Coal \= [],
    majority(Coal),
    pairwise_score(Coal, Score), Score > 0,
    seats_sum(Coal, Seats).
```

- `subset_of` är hela sökningen: för varje parti, ta med det eller inte. Backtracking går igenom alla $2^8 = 256$ delmängder, "ta med" före "hoppa över".
- Villkor 1 är majority: $2*S \geq T$ slipper division; `seats_sum/2` summerar som `total_seats`. "Minst hälften" lästes som $\geq$; strikt majoritet vore $>$.
- Villkor 2 är `pairwise_score`: summan av index inom koalitionen ska vara positiv.
- `stable/3` är mönstret *generera och testa*: en kandidat, sedan filtren, billigast först.

Uppgift 6: vad programmet föreslår

```
?- stable(C, Seats, Score).  
C = [p1,p2,p3,p4,p5,p6,p7,p8],  
Seats = 234, Score = 21 ;  
C = [p1,p2,p3,p4,p5,p6,p7],  
Seats = 209, Score = 16 ;  
...  
  
?- stable(C, Seats, Score), length(C, 4).  
C = [p1,p2,p5,p6], Seats = 117, Score = 17 ;  
C = [p1,p4,p5,p6], Seats = 123, Score = 29 ;  
C = [p1,p4,p6,p8], Seats = 121, Score = 20 ;  
C = [p1,p5,p6,p7], Seats = 118, Score = 14 ;  
C = [p1,p5,p6,p8], Seats = 125, Score = 21 ;  
false.  
  
?- stable(C, _, _), length(C, 3).  
false.
```

- Första svaret är hela parlamentet. Det uppfyller villkoren, men ingen kallar det en regering. "Föreslår" kräver att vi säger vad vi föredrar.
- Med ett storlekskrav efteråt får vi de fem minsta: fyra partier räcker, tre gör det aldrig.
- Sammanlagt finns 63 stabila koalitioner: 5 med fyra partier, 28 med fem, 21 med sex, 8 med sju och parlamentet självt.
- p3 (51 mandat, alla index negativa) ingår i ingen av de små: stort och osams väger mindre än litet och överens.

Uppgift 6: två saker att säga högt

Vanligt fel

Repots ursprungliga `pairwise_score` räknade varje par två gånger: `select` tog ut A, `member` hittade B i resten, och sedan tvärtom. Tecknet blev rätt, summan dubbel: parlamentet fick 42 i stället för 21. Villkoret `Score > 0` märkte inget. Den rättade versionen:

```
pairwise_score(Coal, Score) :-  
    findall(S, (member(A, Coal), member(B, Coal), A @< B, sim(A, B, S)), Ss),  
    sum_list(Ss, Score).
```

Vad "föreslå" betyder

Programmet räknar upp alla stabila koalitioner; att välja en är en policyfråga som inte står i uppgiften. Tre tolkningar, en rad var: först partier (`length(C, K)` med växande K), minsta mandatöverskott, eller högst index. Högst har `[p1,p2,p4,p5,p6,p7,p8]`: alla utom p3, med index 54. Med åtta partier räcker det att prova alla 256 kandidater; "svår" betyder modellering, inte optimering.

Uppgift 7

Skolschema som databas (svår)

Uppgift 7: databasen

Uppgiften ber om två saker: hur fakta ser ut, och *i ord* hur predikaten svarar på frågorna. Koden i `scheduling.pl` är en bonus.

```
teacher(t_alice).    student(s_marcus, cs).    program(cs).
course(da101, 'Declarative Programming').
teaches(t_alice, da101).      % läraren undervisar kursen
enrolled(s_marcus, da101).    % studenten läser kursen

% session(Kurs, Moment, Dag, Start, Slut, Typ, Obligatoriskt); tid i minuter
session(da101, lab1a, tue, 10*60, 12*60, lab1,    yes).
session(da101, lab1b, wed, 14*60, 16*60, lab1,    yes).    % samma moment, ny tid
session(da101, lect1, mon, 9*60, 11*60, lecture, no).
session(da101, exam,  fri, 13*60, 16*60, exam,    yes).
session(pr301, demo1, tue, 11*60+30, 12*60+30, demo, yes).
```

- En relation per fråga i texten: vem undervisar, vem läser, vilket program, när något sker. Tiden är tal så att överlapp blir aritmetik; Typ säger vilket *moment* ett tillfälle tillhör: två lab1-rader är två chanser att gå på samma labb.

Uppgift 7: predikaten, i ord och i kod

```
overlap(Day, S1, E1, Day, S2, E2) :-  
    S1 < E2, S2 < E1.  
  
teacher_conflict(T, C1-Sess1, C2-Sess2) :-  
    teaches(T, C1), teaches(T, C2), C1 \= C2,  
    session(C1, Sess1, D, A, B, _, _),  
    session(C2, Sess2, D, C, E, _, _),  
    overlap(D, A, B, D, C, E).  
  
no_teacher_conflicts :-  
    \+ teacher_conflict(_, _, _).
```

```
% om t_alice också undervisar ma201:  
?- teacher_conflict(T, A, B).  
T = t_alice, A = da101-lab1a,  
B = ma201-help1 ;  
T = t_alice, A = ma201-help1,  
B = da101-lab1a.
```

- **Överlapp:** två intervall samma dag krockar om vart och ett börjar innan det andra slutar. Halvöppna intervall: 10–12 och 12–14 krockar inte.
- **Lärarvillkoret** är ett förbud, så vi letar efter brott mot det: samma lärare, två olika kurser, två tillfällen samma dag som överlappar. Finns inget sådant är schemat OK: negation as failure från uppgift 2 igen. (Varje krock listas åt båda hållen; $C1 @< C2$ tar bort dubbletten.)
- **Studentvillkoret** är inte ett förbud: överlapp får finnas, bara det går att välja ett tillfälle per obligatoriskt moment utan krockar. Nästa bild.

Uppgift 7: studenten och de obligatoriska momenten

```
req(St, C-T) :-          % krav: (kurs, moment)
    enrolled(St, C),
    session(C, _, _, _, T, yes).
requirements(St, Reqs) :-
    ( setof(CT, req(St, CT), Reqs) -> true
    ; Reqs = [] ).
choose([], Acc, Acc).
choose([C-T|Rs], Acc, Out) :-
    session(C, Id, D, A, B, T, yes),
    \+ ( member(_-_-s(D2,A2,B2,_), Acc),
        overlap(D, A, B, D2, A2, B2) ),
    choose(Rs, [C-T-s(D,A,B,Id)|Acc], Out).
feasible(St, Choice) :-
    requirements(St, Reqs),
    choose(Reqs, [], Choice).
student_ok(St) :- feasible(St, _).
```

```
?- requirements(s_marcus, R).
R = [da101-exam, da101-lab1,
     ma201-exam, pr301-demo].
?- feasible(s_marcus, C).
C = [pr301-demo-s(thu,15*60,16*60,demo2),
     ma201-exam-s(thu,13*60,15*60,exam),
     da101-lab1-s(tue,10*60,12*60,lab1a),
     da101-exam-s(fri,13*60,16*60,exam)] ;
...      % tre lösningar totalt
```

- setof samlar kraven som en sorterad mängd utan dubletter.
- choose: generera och testa igen. session ger ett tillfälle i taget, \+ kollar att det inte krockar med de valda, rekursionen tar nästa krav; krockar allt backtrackar Prolog.
- student_ok frågar bara om det finns *någon* lösning.

Uppgift 7: fallgropar, och vad tentan vill ha

- **Fria variabler i setof.** Repots `student_requirements` skriver `session(Course, S, _, _, _, T, yes)` direkt inne i `setof` och kvantifierar bara `S` med `^`. De tre `_` är då fria, `setof` grupperar svaren per (dag, start, slut), och det efterföljande `!` behåller första gruppen: resultatet blev `[da101-exam]` i stället för fyra krav. Lägg målet i ett eget predikat (`req/2` ovan) eller kvantifiera alla hjälpvariabler.
- **Oevaluerad tid.** `10*60` lagras som en term. `<` räknar ut den, så jämförelserna fungerar, men svaren skrivs ut som `10*60`. Lagra 600 om utskriften ska läsas av människor.
- **Program.** `student(Namn, Program)` täcker "vilket program de tillhör"; en fråga som "vilka studenter från cs läser da101" är då ett `enrolled` plus ett `student`.

Poängen

Uppgiften bedöms på designen: rätt fakta med rätt argument, och en förklaring av hur varje fråga besvaras genom att gå igenom fakta, testa villkor och backtracka. En student som skriver "överlapp = börjar innan den andra slutar, åt båda hållen" och "läarvillkoret = det finns inget motexempel" har svarat, även utan en rad kod.

Uppgift	Det som ska sitta
1 Unifiering	= binder permanent; ett mål som inte går att unifiera faller hela frågan: <code>false</code>
2 Negation	<code>\+ G = "G går inte att bevisa"</code> ; negerar bara bundna mål; inga bindningar ut
3 remove last	en klausul per fall i listdefinitionen; basfallet är <code>[_]</code> , inte <code>[]</code>
4 Träd	data är termer; strukturell rekursion med en klausul per funktor; <code>is</code> för aritmetik
5 permute	säg vad en permutation är, låt <code>select</code> och backtracking generera alla $n!$
6 Filurien	generera (<code>subset_of</code>) och testa (<code>majority</code> , <code>pairwise_score</code>); säg vad "föreslå" betyder
7 Schema	fakta med rätt argument; villkor som "inget motexempel" eller "det finns ett val"

Kör själva: `swipl filurien.pl` och sedan `?- stable(C, S, Sc), length(C, 4)`. Hela repot: github.com/dicander/prolog.